



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn

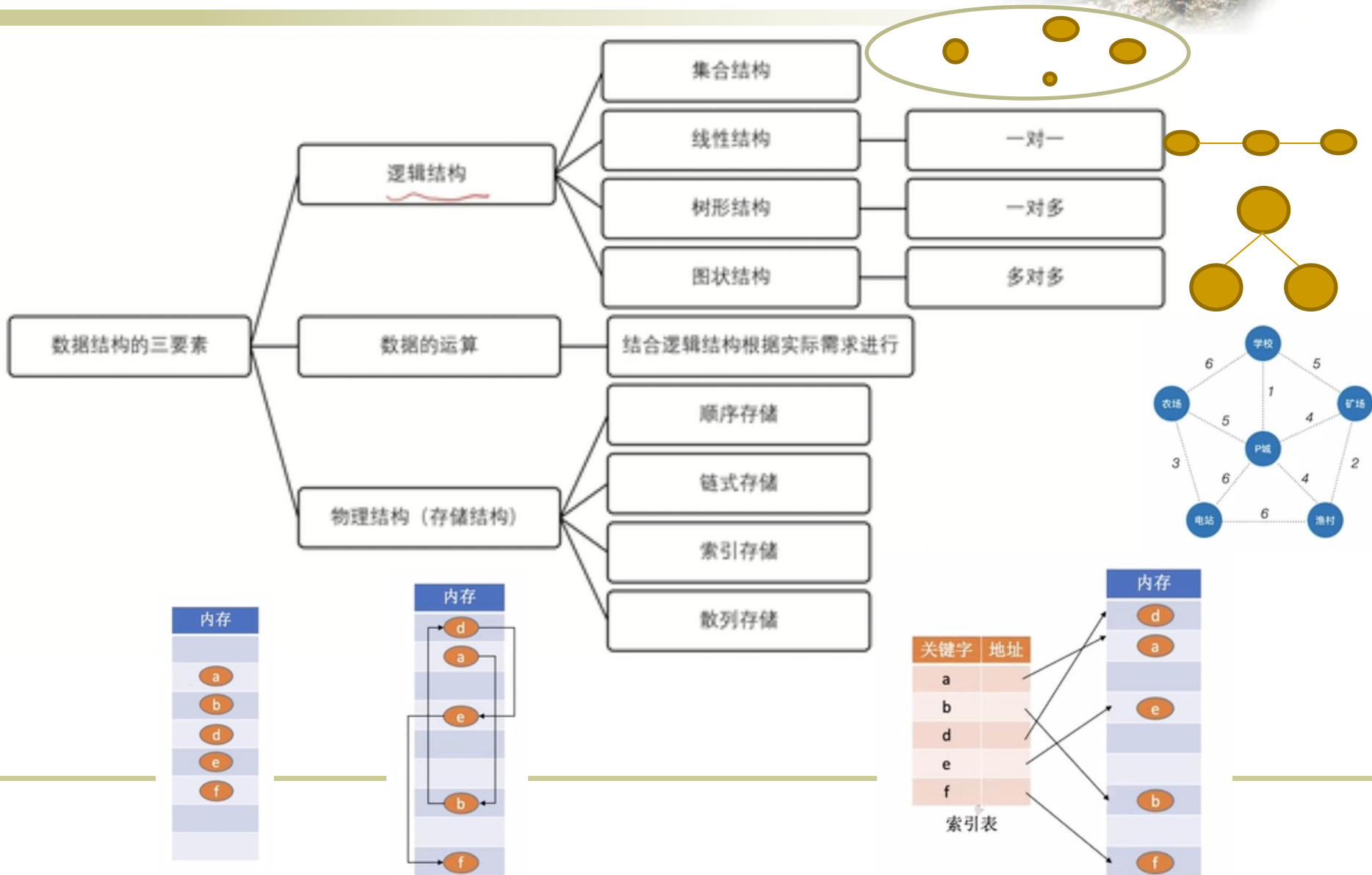
南京大学计算机学院



第十章 文件、外部排序 与外部搜索



- 主存储器和外存储器
- 文件组织
- 多级索引结构
- 外排序





存储器基础知识

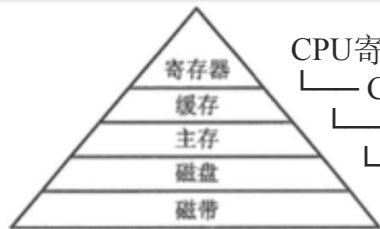
概述

是什么
核心问题

存放信息
打游戏 背包与仓库

高速设备如何适配慢速设备

寄存器(CPU) 缓存(Cache) 主存(主机) 辅存(I/O设备)



CPU寄存器 (Register) < 1KB ≈ 0.3 ns
CPU缓存 (Cache) L1~L3 1-10 ns (SRAM)
主存储器 (RAM) 4-128GB 80-100 ns (DRAM)
外存储器 (Disk) 1-100TB 5-10 ms (SSD/HDD)
归档存储 PB级 秒-分 (磁带/光盘)

图 4.2 存储器速度、容量和位价的关系

越往下，容量越大、
速度越慢、单位成本越低

三种存储方式

17



图 4.1 存储器分类

三种存储方式

RAM Random Access Memory

随机存取存储器

速度都相同

分类

DRAM

用作主存

动态

SRAM

用作缓存

静态static

SAM

sequential

顺序存取存储器

速度取决于物理位置

DAM

direct

直接存取存储器

兼并上述二者



关键性能指标对比表



存储器类型	访问时间	容量	成本/GB	易失性	典型用途
寄存器	0.3 ns	< 1KB	-	易失	指令操作数
缓存	1-10 ns	MB级	高	易失	缓存热点数据
主存	80-100 ns	GB级	中	易失	程序运行空间
SSD	0.1-1 ms	TB级	较低	非易失	系统盘/数据库
HDD	5-15 ms	TB级	低	非易失	数据存储
磁带	秒级	PB级	极低	非易失	归档备份



10.1 主存储器与外存储器



- 主存储器又叫内存储器，简称为内存；外存储器简称为外存。
- 外存储器与内存储器相比，优点是：
 - ◆ 价格较低
 - ◆ 永久的存储能力
- 缺点：
 - ◆ 访问外存储器上的数据比访问内存要慢5~6个数量级
- 要求我们在开发系统时必须考虑如何使外存访问次数达到最少。



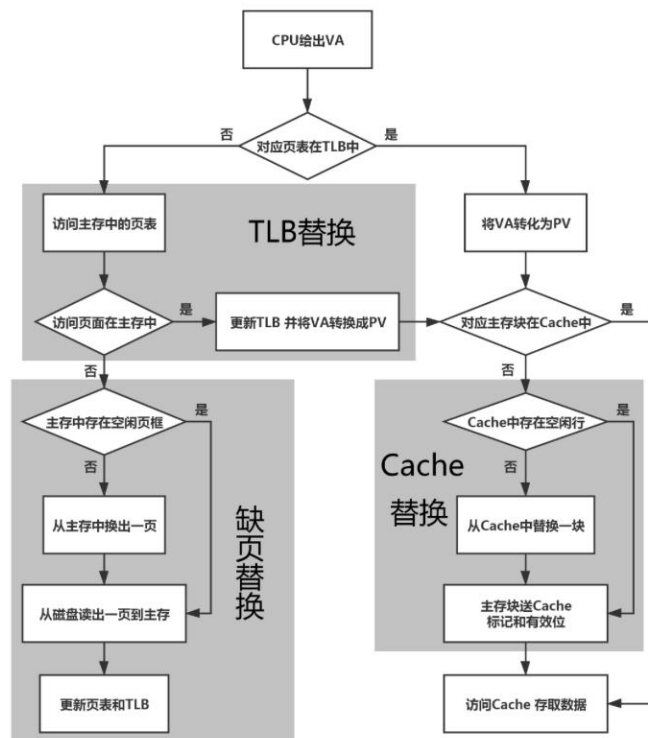
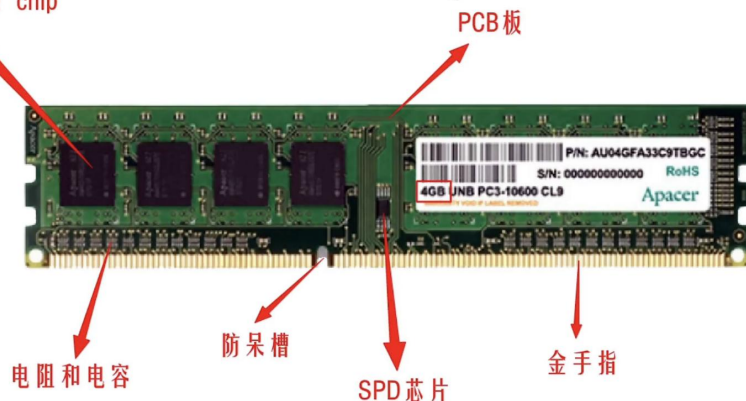
主存储器 (RAM) 详解



1. 物理结构：芯片级视图

2. 内存访问过程

内存颗粒也叫内存芯片 英文名: chip 一个内存芯片的容量: $4096\text{MB} \div 8 = 512\text{MB}$



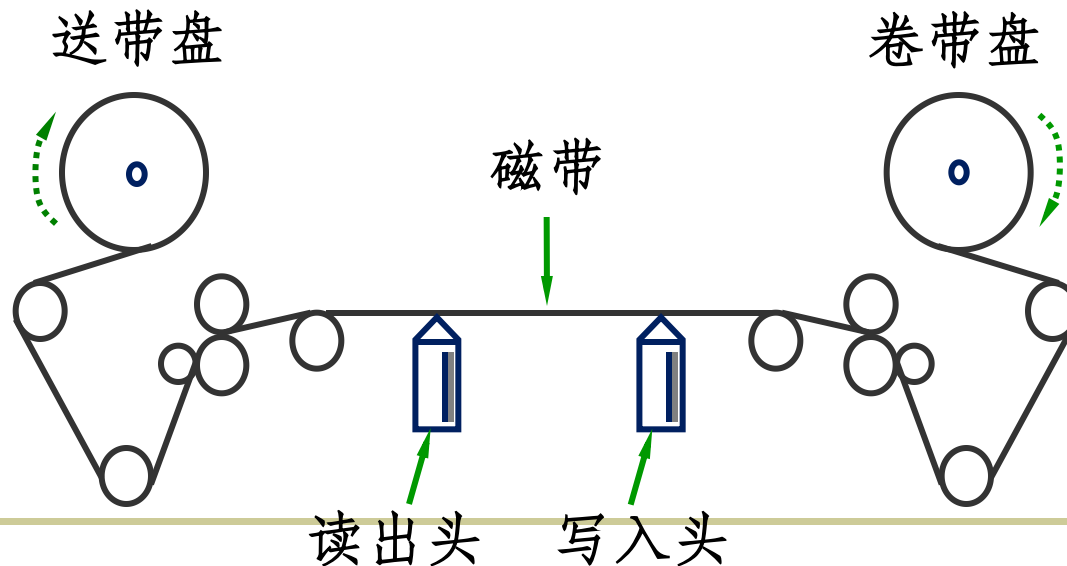
注释：
VA (virtual address) : 虚拟地址 (逻辑地址)
PV (physical address) : 实际地址 (物理地址)
TLB (Translation Lookaside Buffer) : 转换检测缓冲区



10.1.1 磁带 (tape)



- 磁带是一种**顺序存取**设备。
- 磁带主要用于备份、存储不经常使用的数据，以及作为将数据从一个系统转移到另一个系统的脱机介质。







- 磁带卷在一个卷盘上，运行时磁带经过读写磁头，把磁带上的信息读入计算机，或者把计算机中的信息写到磁带上。
- 数据记录在磁带带面上。在带面上并列存放有 9 个磁道的信息，即**每一横排有 9 位二进制信息**：8 位数据加 1 位奇偶校验位。
- 磁带的存储密度用 BPI (Bit Per Inch) 为单位，典型的存储密度有 3 种：6250BPI (=246排/mm)、1600BPI (=64排/mm)、800BPI (32排/mm)。正常走带速度为 3 ~ 5m/Sec，因设备而异。

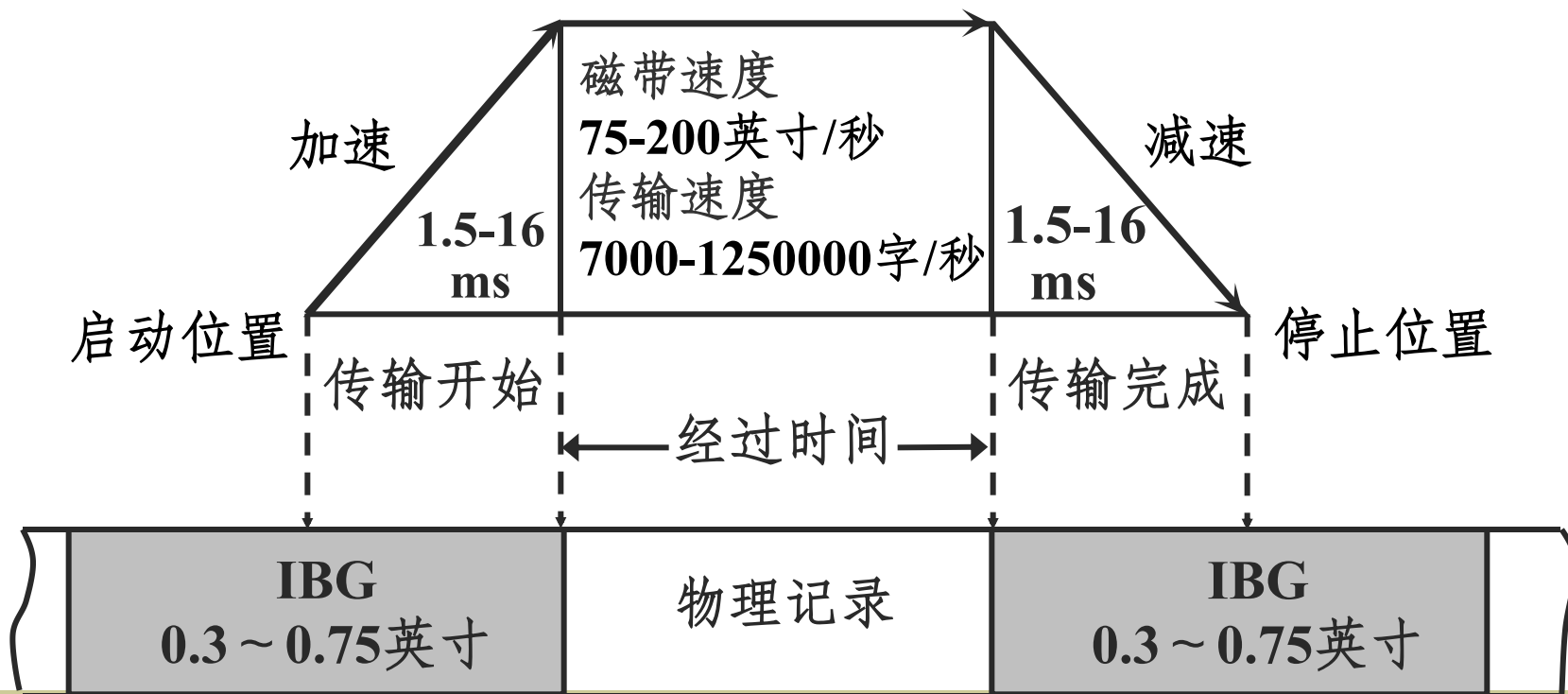


- 数据的传送速度 = 存储密度 × 走带速度。
- 在应用中使用文件进行数据处理的基本单位叫做逻辑记录，简称为记录；在磁带上物理地存储的记录叫做物理记录。
- 在使用磁带或磁盘存放逻辑记录时，常常把若干个逻辑记录打包进行存放，把这个过程叫做“块化”（blocking）。经过块化处理的物理记录叫做块化记录。
- 磁带设备是一种启停设备。磁带每次启停都有一个加速与减速的过程，在这段时间内走带不稳定，只能走空带，这段空带叫做记录¹¹



间间隙IRG（Inter Record Gap）或者块间间隙IBG（Inter Block Gap），其长度因设备而异。

定速





如果每个逻辑记录是 80 个字符，IRG 为 0.75 英寸，则对存储密度为 1600BPI 的磁带，一个逻辑记录仅占 $80/1600 = 0.05$ 英寸。每传输一个逻辑记录磁带走过 0.05 英寸，接着磁带要走过一个 IRG 占 0.75 英寸。结果大部分时间都花费在走空带上，存储利用率只有 1/16。

- 如果将若干逻辑记录存放于一个块，将 IRG 变成 IBG，可以提高存储利用率。例如，将 50 个有 80 个字符的逻辑记录放在一个块内，此块的长度将达到 $50 \times 80 / 1600 = 2.5$ 英寸，存储利用率达到 0.77。因此在磁带上采用 **按块读写**。



- 在磁带设备上读写一块信息所用时间

$$t_{IO} = t_a + t_b$$

- 其中， t_a 是延迟时间，即读写磁头到达待读写块开始位置所需花费的时间，它与当前读写磁头所在位置有关。 t_b 是对一个块进行读写所用时间，它等于数据传输时间加上**IBG**时间。
- 磁带设备只能用于处理变化少，只进行**顺序存取**的大量数据。

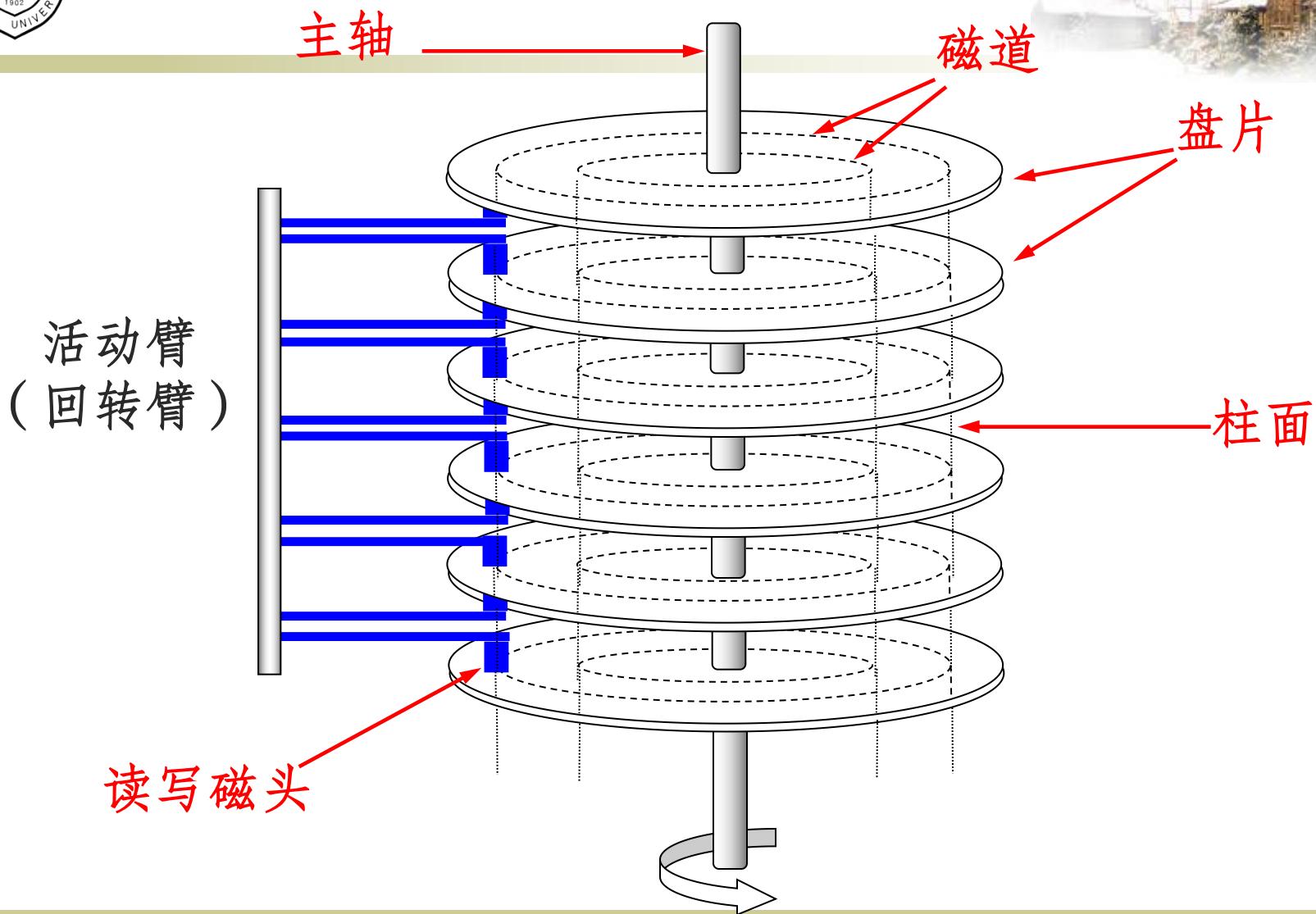




10.1.2 磁盘 (disc)



- 磁盘存储器通常称为直接存取设备，或随机存取设备，它访问外存上文件的任一记录的时间几乎相同。
- 磁盘存储器可以顺序存取，也可以随机存取。
- 目前使用较多的是**活动臂硬盘组**：若干盘片构成磁盘组，它们安装在主轴上，在驱动装置的控制下高速旋转。
- 除了最上面和最下面一个盘片的外侧盘面不用以外，其他每个盘片上下两面都可存放数据。将这些可存放数据的盘面称为**记录盘面**。

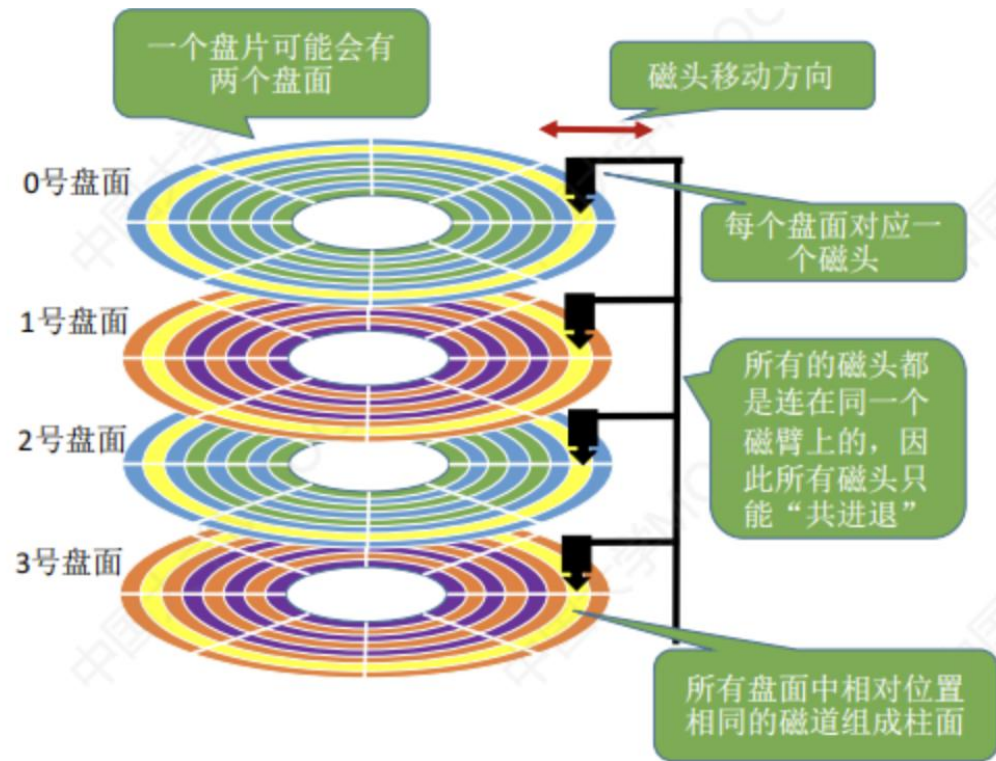




- 每个记录盘面上有很多**磁道**，数据就存放在这些磁道上，在记录盘面上形成一个个同心圆。
- 每个记录盘面都有一个读写磁头。所有记录盘面的读写磁头都安装在同一个动臂上，随动臂向内或向外做径向移动，从一个磁道移到另一个磁道。
- 任一时刻，所有记录盘面的读写磁头停留在各个记录盘面的**半径相同**的磁道上。
- 运行时，由于盘面做高速旋转，磁头所在的磁道上的数据相继在磁头下，从而可读写数据。



- 各个记录盘面上半径相同的磁道合在一起称为**柱面**。动臂的移动实际上是将磁头从一个柱面移动到另一个柱面上。
- 一个磁道可以划分为若干段，称为**扇区**，一个扇区就是一次读写的最小数据量。
- 对磁盘存储器来说，从大到小的存储单位是：**盘片组、柱面、磁道和扇区**。





- 对磁盘存储器进行一次存取所需时间：
 1. 当有多个盘片组时，要选定某个盘片组。这是由电子线路实现的，速度很快。
 2. 选定盘片组后再选定某个柱面，并移动动臂把磁头移到此柱面上。这是机械动作，速度较慢。这称为“寻查（seek）”。



3. 选定柱面后，要进一步确定磁道，即确定由哪个读写磁头读写，由电子线路实现。
4. 确定磁道后，还要确定所要读写数据在磁盘上的位置（如在哪一个扇区）。这实际上就是在等待要读写的扇区转到读写磁头下面。这是机械动作。这段时间一般称为旋转延迟（rotational delay）时间。
5. 真正进行读写时间。



- 在磁盘组上一次读写的时间主要为：

$$t_{io} = t_{seek} + t_{latency} + t_{rw}$$

- 其中， t_{seek} 是平均寻查时间，是把磁头定位到要求柱面所需时间，这个时间的长短取决于磁头移过的柱面数。
- $t_{latency}$ 是平均等待时间，是将磁头定位到指定块所需时间。
- t_{rw} 是传送一个扇区数据所需的时间。

读12扇区



寻求正确的轨道。磁头当前所在的位置在外围的21扇区，如果要访问12，必须先将磁头移到其所在的位置，即寻道（seek^Q）。平均寻道时间大约需要 4ms。完全寻道（从表面的一端到另一端）可能需要两到三倍的时间。寻道有许多阶段：首先是磁盘臂移动时的加速阶段；然后随着磁盘臂全速移动而惯性滑动；然后随着磁盘臂减速而减速；最后，在磁头小心地放置在正确的磁道上时停下来。不是纯粹的线性成本。必须加速、滑行、减速、停放。停放时间（settling time）可能需要 0.5~2ms，因为驱动器必须确定找到正确的磁道。

等待旋转。等待期望的扇区旋转到磁头下。这种等待在现代驱动器中经常发生，并且是 I/O 服务时间的重要组成部分，又称之为旋转延迟（rotational delay/rotation delay）。平均旋转延迟直接根据 RPM 计算。15000 RPM 等于 250 RPS（每秒转速）。因此，每次旋转需要 4ms。平均而言，磁盘将会遇到半圈旋转，因此平均时间为 2ms。

传输数据。传输速度很快，取决于 RPM 和扇区密度。100+ MB/s 是典型的最大传输速率。传输时间就是传输大小除以峰值传输速率。512 字节 * (1s / 100 MB) $\approx$ 5 us。

读/写时间由三个组成部分：时间 = 寻道 + 旋转 + 传输时间



- 在MS-DOS系统中，多个扇区集结成组，称为簇。簇是文件分配的最小单位，其大小由操作系统决定。
- 在UNIX系统中，文件分配的最小单位和读写的最小单位是一个扇区，称为一个块block。
- 磁盘一次读写操作访问一个扇区，称为访问“一页”（page）或“一块”（block），又称为“一次访外”。



10.1.3 缓冲区 (buffer)



- 为了实施磁盘读写操作，在**内存中**需要开辟一些区域，用以存放需要从磁盘读入的信息，或存放需要写出的信息。这些内存区域称为**缓冲区**。多数操作系统至少设置两个缓冲区，一个为输入缓冲区，一个为输出缓冲区。
- 例如，在从磁盘向内存读入一个扇区的数据时，数据被存放到输入缓冲区，如果下次需要读入同一个扇区的数据，就可以直接从缓冲区中读取数据，不需要重新读盘。



- 磁盘缓冲区是位于磁盘与内存之间的一种数据缓冲区，用于解决速度不匹配的问题，减少直接访问的灾难。
- 通常由动态随机访问存储器（**DRAM**）构成，作为内存的一部分被分配给磁盘使用，以临时存储从磁盘读取或即将写入磁盘的数据块。
- 这一设计旨在减少**CPU**直接访问磁盘的次数，因为**DRAM**的读写速度远高于磁盘，从而有效提升数据处理的效率。



■ 磁盘缓冲区的作用

- 1. 提高读取效率：磁盘缓冲区能够显著缩短数据从磁盘到主内存的传输时间。
- 2. 提升**CPU**性能：由于**CPU**可以直接从磁盘缓冲区读取数据，而无需等待磁盘旋转和寻道，因此**CPU**可以更高效地执行其他任务。
- 3. 优化数据访问顺序：磁盘缓冲区能优化数据访问顺序，频繁访问的数据会被保留在缓冲区中，减少磁盘的寻道时间和旋转延时。



- 缓冲区大小应与操作系统一次读写的块的大小相适应，这样可以通过操作系统一次读写把信息全部存入缓冲区中，或把缓冲区中的信息全部写出到磁盘。
- 如果缓冲区大小与磁盘上的块大小不适配，就会造成存储空间的浪费。
- 缓冲区的构造可以看作一个先进先出的队列。



缓冲区的定义及其操作



```
#include <iostream.h>
```

```
#include <assert.h>
```

```
const int DefaultSize = 2048;
```

```
template <class T>
```

```
struct buffer {
```

```
    T *data;                //缓冲区数组
```

```
    int current, maxSize;   //当前指针, 缓冲区容量
```

```
    buffer (int sz = DefaultSize) : maxSize(sz), current(0)
```

```
    { data = new T[sz]; assert (data != NULL); }
```

```
    ~buffer() { delete []data; }
```



```
void OutputInfo (ostream& out, T x); //缓冲区输出
void InputInfo (istream& in, T& x); //缓冲区输入
};

template <class T>
void buffer<T>::OutputInfo (ostream& out, T x) {
    if (current == maxSize) {
        for (int i = 0; i < maxSize; i++) out << data[i];
        current = 0;
    }
    data[current] = x; current++;
};
```




```
template <class T>
void buffer<T>::InputInfo (istream& in, T& x) {
    if (current < maxSize) {
        x = data[current];
        current++;
    }
    else {
        for (int i = 0; i < maxSize; i++) in >> data[i];
        current = 0;
    }
};
```



缓冲区替换算法



■ 当缓冲区满时，需要替换出一些块：

(1) LRU（最近最少使用） (2) Clock算法（二次机会算法）

// 每次访问时，将块移到链表头部

```
BufferBlock* access_block(BufferPool* pool, int block_id) {  
    BufferBlock* block = find_block(pool, block_id);  
    if (block) {  
        // 从LRU链表中移除  
        remove_from_lru(block);  
        // 插入到链表头部（表示最近使用）  
        insert_to_lru_head(pool, block);  
    }  
    return block;  
}
```

// 需要替换时，选择链表尾部的块（最久未使用）

```
BufferBlock* victim = pool->lru_tail;
```

循环扫描缓冲区，给每个块一个“使用位”

访问块时：使用位置1

替换时：扫描，使用位1→置0继续，使用位0→替换
类似时钟指针循环



1. 下列存储设备中，属于主存储器的是：

- A) 固态硬盘(SSD)
- B) 光盘(CD-ROM)
- C) 随机存取存储器(RAM)
- D) 磁带

■ 正确答案： C

■ 解析： 主存储器（内存）包括RAM和ROM，直接与CPU交换数据。固态硬盘、光盘和磁带都属于外存储器，需要通过I/O接口与内存交换数据。



2. 关于外存储器的特点，下列说法错误的是：

- A) 存储容量通常比主存储器大
- B) 存取速度通常比主存储器慢
- C) 断电后数据会丢失
- D) 包括硬盘、U盘、光盘等设备

■ 正确答案： C

■ 解析： 外存储器（如硬盘、U盘）是永久性存储设备，断电后数据不会丢失。而主存储器中的 **RAM** 是易失性存储器，断电后数据会丢失。



- 1. 主存储器主要由 _____ 和 _____ 组成，其中 _____ 是易失性存储器，断电后数据会丢失。
- 正确答案： RAM、ROM、RAM
- 2. CPU可以直接访问 _____ 存储器，而不能直接访问 _____ 存储器，后者中的数据需要先加载到前者中才能被CPU处理。
- 正确答案： 主（或内）、外



10.2 文件组织



10.2.1 文件的基本概念

- 什么是文件
 - ◆ 文件是存储在外存上的数据结构，一般是在逻辑上具有完整意义的一组相关信息项的有序序列。
 - ◆ 文件分操作系统文件和数据库文件
 - ✓ **操作系统文件**是流式文件：没有结构的字符流
 - ✓ **数据库文件**是具有结构的数据集合
 - ◆ 数据结构中讨论的是数据库文件。
- 操作系统对文件是按物理记录读写的；
- 在数据库中文件按页块存储和读写。



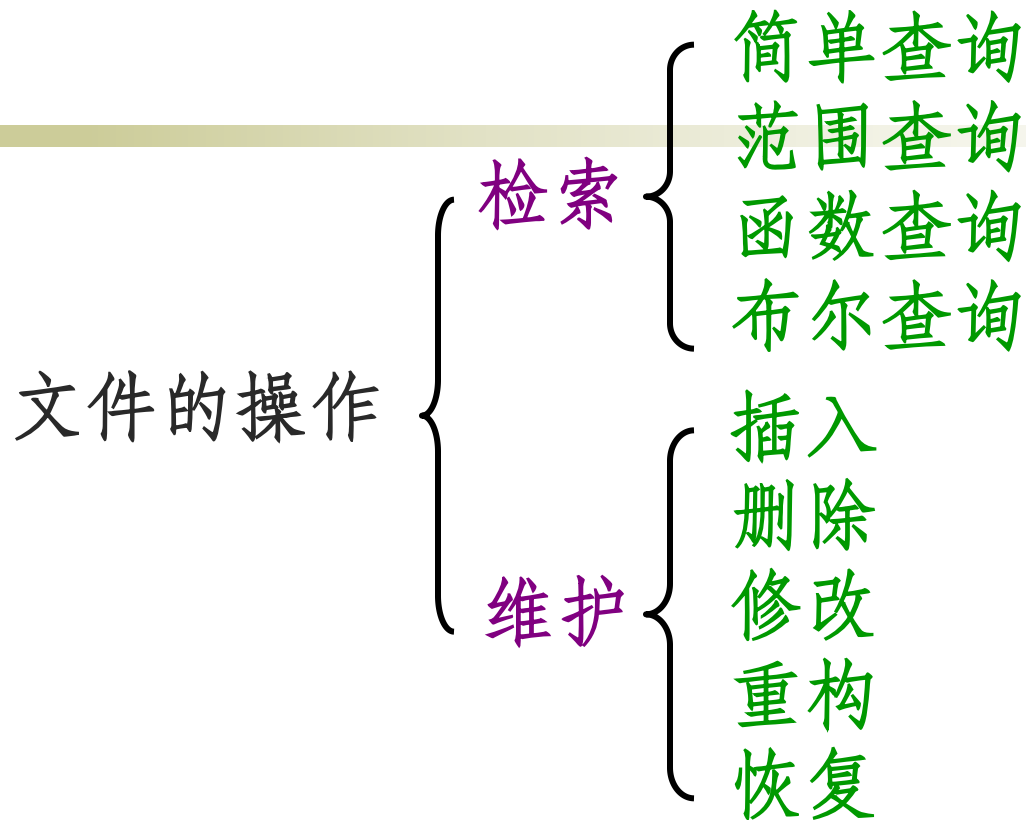
文件的组成



- 文件由记录组成；记录由若干数据项组成。
- 记录是文件存取的基本单位，数据项是文件可使用的最小单位。
- 从不同的观点，文件记录分为逻辑记录和物理记录。前者是面向用户的基本存取单位，后者是面向外设的基本存取单位。
- 能够唯一标识一个记录的数据项或数据项集称为主关键码项，其值称为主关键码；
- 不唯一标识一个记录的数据项或数据项集称为次关键码项，其值称为次关键码。



- 文件结构包括文件的**逻辑结构**、文件的**存储结构**和文件的**操作**。
- 文件的逻辑结构是**线性结构**，各个记录以线性方式排列。
- 文件的**存储结构**是指文件在外存上的组织方式，它与文件特性有关。
 - ◆ 顺序组织
 - ◆ 直接存取组织（散列组织）
 - ◆ 索引组织
- 文件的操作是**定义在逻辑结构上**的，但操作的**具体实现**要在**存储结构**上进行。



- 评价一个文件组织的效率
 - ◆ 执行文件操作所花费的时间
 - ◆ 文件组织所需要的空间



1.顺序文件 (Sequential File)



- 顺序文件中的记录按它们进入文件的先后顺序存放，其逻辑顺序与物理顺序一致。
- 如果文件的记录按主关键码有序，则称其为顺序有序文件，否则称其为顺序无序文件。
- 顺序文件通常存放在顺序存取设备（如磁带）上或直接存取设备（如磁盘）上。
 - 当存放在顺序存取设备上时，只能按顺序搜索法存取；
 - 当存放在直接存取设备上时，可以使用顺序搜索法、折半搜索法等存取。



● 顺序文件的存储方式

- ✓ 连续文件：文件的全部记录顺序地存放于外存的一个连续的区域中。
 - 优点是存取速度快、存储利用率高、处理简单。
 - 缺点是区域大小需事先定义，不能扩充。
- ✓ 串联文件：文件记录成块存放于外存中，在块中记录顺序连续存放，但块与块之间可以不连续，通过块链指针顺序链接。
 - 优点是文件可以扩充、存储利用率高。
 - 缺点是影响了存取和修改的效率。



2.直接存取文件 (Direct Access File)



- 又叫**散列文件**。利用散列技术组织文件。处理类似散列法，但它是存储在外存上的。
- 文件记录的逻辑顺序与物理顺序不一定相同。通过记录的**关键码**可直接确定该记录的地址。
- 使用**散列函数**把关键码集合映射到地址集合时，往往会产生地址冲突，处理冲突有两种处理方式：
 - ◆ 按桶散列
 - ◆ 可扩充散列



(1) 按桶散列



- 文件中的记录成组存放，若干个记录组成一个存储单位，称之为桶。假若一个桶能存放 m 个记录，则 m 个互为同义词的记录可以存放在同一地址的桶中。当第 $m+1$ 个同义词出现时，才发生“溢出”。

(a) 溢出链

- 当发生“溢出”时，将第 $m+1$ 个同义词存放到“溢出桶”。并称存放前 m 个同义词的桶为“基桶”。溢出桶和基桶大小相同。当在基桶中检索不成功，就循指针到溢出桶中检索。



桶大小为3的溢出桶链表示例

基桶编号 基桶区

溢出桶编号 溢出桶区

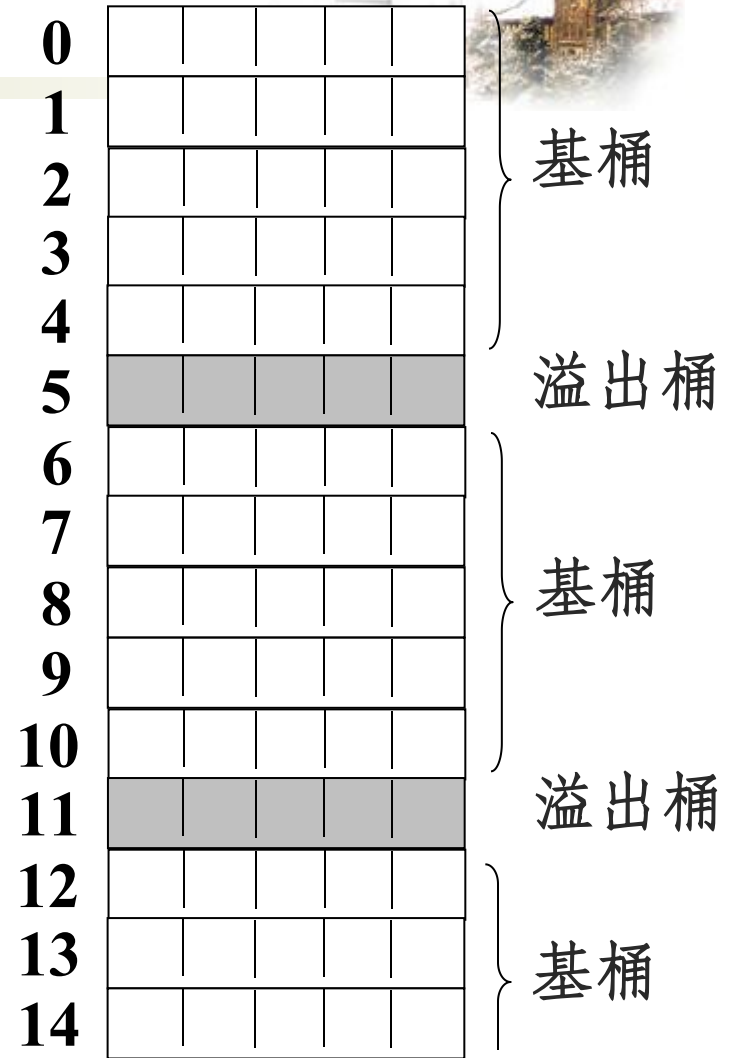
0	070			Λ
1	512	204	246	O ₁
2	597	177		Λ
3	262	157		Λ
4	116	613		Λ
5	285	635	208	O ₂
6	923	076		Λ

O ₁	015	337	988	O ₃
O ₂	817	117	390	O ₄
O ₃	575	540	435	Λ
O ₄	362			Λ
O ₅				
O ₆				
O ₇				

- 在这种散列文件中删除记录时，因为可能需要重新链接，所以只需做一个逻辑删除标记即可，待系统做周期性重构时再做物理删除。

(b) 分布式溢出空间

- ◆ 溢出桶按照一定的间隔分布在基桶之间。如果有一个基桶溢出了，系统就将记录存放在下一个溢出桶中。
- ◆ 如果溢出桶自己溢出了，则使用下一个相继的溢出桶，这需要第二次溢出处理。



分布式溢出桶



- 如果系统对基桶按0, 1, 2, 3, 4, 5, ...进行编号, 在按间隔 $G = 5$ 插入溢出桶后, 可按下列公式按字节求出各个桶的实际存储地址:

$$\text{桶的地址} = \mathbf{B}_0 + \mathbf{B} \times \left(\mathbf{i} + \left\lfloor \frac{\mathbf{i}}{5} \right\rfloor \right).$$

- 其中, $\mathbf{B}_0$ 是在文件中第0号桶的起始地址, $\mathbf{B}$ 是每个桶的字节数。
- 在括号中的除数5, 表示每隔5个基桶安排一个溢出桶。



$$H(\text{key}) = \text{key} \% 11$$

(c) 相继溢出法

- ◆ 此方法 **不设置溢出桶**。当记录应存放的桶溢出时，溢出记录存放 to 下一个相继的桶中。
- ◆ 如果该桶已满，就把它放到再下一个桶中，如此处理，直至把记录存放好。
- ◆ 相继溢出法的优点是对溢出不需要漫长的寻找。紧邻的桶通常相距不多于一次磁盘旋转。但当邻近的多个桶被挤满时，则为了查找空闲空间就需要检查许多桶。如果桶的容量很小更是如此。

0	362			
1	177			
2				
3	597	157	817	575
4	070	246	015	542
5	389			
6	116	204	512	435
7	337	117		
8	635	613		
9	262	988		
10	285	923	076	208

相继溢出法



(2) 可扩充散列



- 这是基于数字搜索树的一种散列方法，细节参见10.5节。
- 不作要求



散列文件优缺点



- **散列文件**具有随机存放、记录不需进行排序、插入删除方便、存取速度快、不需要索引区和节省存储空间等优点。
- 散列文件不能顺序存取，只能按关键码随机存取。
- 在经过多次插入、删除后，可能出现溢出桶满而基桶内多数记录已被删除的情况。此时需要重新组织文件。



3. 索引文件 (Indexed File)



- 索引文件由索引表和数据表（主文件）组成。
- 索引表用于指示逻辑记录与物理记录间的对应关系，它是按关键码有序的表。
 - ◆ 索引顺序文件：主文件也按关键码有序。此时可对主文件分组，一组记录对应一个索引项。称这种索引表为**稀疏索引**。
 - ◆ 索引非顺序文件：主文件中记录未按关键码有序。此时，每一个主文件记录必须对应一个索引项。称这种索引表为**稠密索引**。



- 静态索引：采用多级索引结构，每一级索引均为有序表。优点是结构简单，缺点是修改很不方便，每次修改都要重组索引。
- 动态索引：采用可动态调整的平衡搜索树结构，如二叉搜索树、B树与B+树等。优点是插入、删除和搜索都很方便。
- 在文件中搜索时，访问外存所花费时间比在内存中搜索所需的时间大得多，因此，外存上搜索一个记录的时间代价主要取决于访问外存的次数，即索引树的高度。



索引非顺序文件示例

索引表

数据表

key	addr
03	2k
08	1k
17	6k
24	4k
47	5k
51	7k
83	0
95	3k

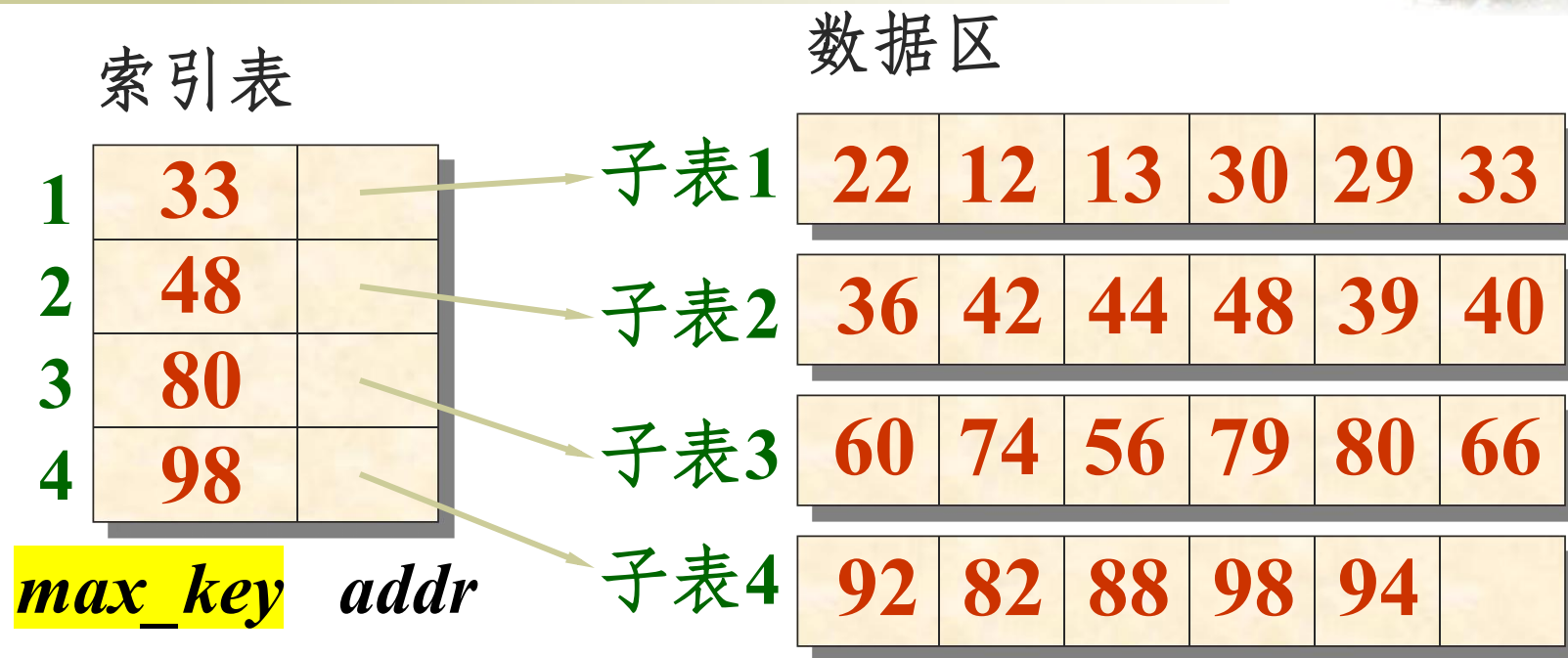
Diagram showing mapping from Index Table to Data Table:

- 03 → 2k
- 08 → 1k
- 17 → 6k
- 24 → 4k
- 47 → 5k
- 51 → 7k
- 83 → 0
- 95 → 3k

职工号	姓名	性别	职务	婚否	...
83	张珊	女	教师	已婚	...
08	李斯	男	教师	已婚	...
03	王璐	男	教务员	已婚	...
95	刘琪	女	实验员	未婚	...
24	岳跋	男	教师	已婚	...
47	周斌	男	教师	已婚	...
17	胡江	男	实验员	未婚	...
51	林青	女	教师	未婚	...



索引顺序文件示例



- 当记录在外存中有序存放时，可以把所有 n 个记录分为 b 个子表(块)存放，一个索引项对应数据表中一组记录(子表)。



- 对索引顺序文件进行搜索，一般分为两级：
 - ◆ 先在索引表 ID 中搜索给定值 K , 确定满足 $ID[i-1].max_key < K \leq ID[i].max_key$ 的 i 值, 即待查记录可能在的子表的序号。
 - ◆ 然后再在第 i 个子表中按给定值搜索要求的记录。
- 索引表是按 max_key 有序的, 且长度也不大, 可以折半搜索, 也可以顺序搜索。
- 各子表内各个记录如果也按关键码有序, 可以采用折半搜索或顺序搜索; 如果不是按关键码有序, 只能顺序搜索。



- 索引顺序文件的搜索成功时的平均搜索长度

$$ASL_{IndexSeq} = ASL_{Index} + ASL_{SubList}$$

- 其中, ASL_{Index} 是在索引表中搜索子表位置的平均搜索长度, $ASL_{SubList}$ 是在子表内搜索记录位置的搜索成功的平均搜索长度。
- 设把长度为 n 的表分成均等的 b 个子表, 每个子表 s 个记录, 则 $b = \lceil n/s \rceil$ 。又设表中每个记录的搜索概率相等, 则每个子表的搜索概率为 $1/b$, 子表内各记录的搜索概率为 $1/s$ 。
- 若对索引表和子表都用顺序搜索, 则索引顺序搜索的搜索成功时的平均搜索长度为

$$ASL_{IndexSeq} = (b+1)/2 + (s+1)/2 = (b+s)/2 + 1$$



- 索引顺序文件的平均搜索长度与表中的记录个数 n 有关，与每个子表中的记录个数 s 有关。在给定 n 的情况下， s 应选择多大？
- 用数学方法可导出，当 $s = \sqrt{n}$ 时， $ASL_{IndexSeq}$ 取极小值 $\sqrt{n} + 1$ 。这个值比顺序搜索强，但比折半搜索差。但如果子表存放在外存时，还要受到页块大小的制约。
- 若采用折半搜索确定记录所在的子表，则搜索成功时的平均搜索长度为

$$\begin{aligned} ASL_{IndexSeq} &= ASL_{Index} + ASL_{SubList} \\ &\approx \log_2(b+1) - 1 + (s+1)/2 \\ &\approx \log_2(1+n/s) + s/2 \end{aligned}$$